

A FIELD GUIDE FOR COMPLETE BEGINNERS

The ROS 2 *Cheatsheet.*

The only ROS 2 basics you need before getting started, the commands you will type every day, and the seven projects that took me from a complete beginner to proficient.

PROJECTS	MONTHS	EARNED	PRIOR EXPERIENCE
24	11	\$10k+	0 yrs

02 What ROS 2 is, and when you need it

03 Nodes, topics, services, actions

04 Packages and workspaces

05 The CLI commands you'll actually use

06 Tools, hardware and setup

07 The visual roadmap

08 Projects 0–2: move, teleop, map

09 Projects 3–6: explore, see, talk, grasp

10 Learning on demand + every link

What ROS 2 is, *and when you need it.*

ROS, the Robot Operating System, is by far the most widely used tool in robotics. NASA picked ROS 2 for the ground software of its VIPER Moon rover. And all you need to start is a laptop.

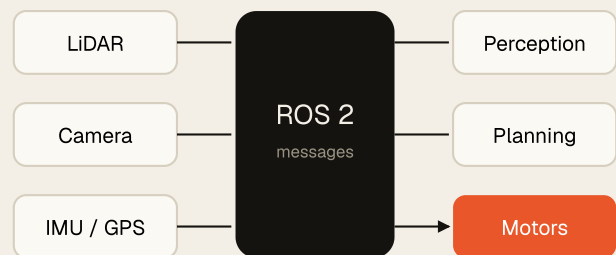
The problem it solves

Picture a self-driving car: LiDAR and cameras to see, a computer to decide, motors to steer, accelerate and brake.

Every sensor speaks at its own rate and in its own format. A LiDAR produces hundreds of thousands of 3D points a second; a camera sends around 30 images a second. All of it has to be processed in parallel and turned into commands.

Without ROS you would write a whole messaging system before a single line of driving logic. And if it all lives in one giant program, one bug in the camera code takes everything down.

ROS is the reliable communication layer between all the parts of your robot, so you can work on higher-level logic instead of plumbing.



Each box is its own program. If one crashes, the rest keep running.

Learn ROS 2, not ROS 1

ROS 1 reached end of life in May 2025. ROS 2 is the version to learn today.

USE ROS 2 WHEN

- ☐ Your robot has several sensors and motors
- ☐ You need simulation before touching hardware
- ☐ You want ready-made navigation (Nav2) or arm planning (MoveIt)
- ☐ Several programs must share data in real time

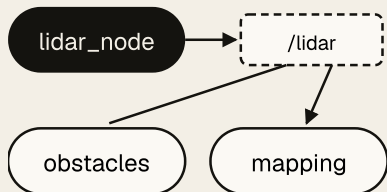
SKIP IT WHEN

- The project is simple, like a line-following robot
- An Arduino sketch can do the whole job
- There is one sensor, one motor and no simulation

Nodes talk in *three ways*.

Each program in ROS 2 is a **node**, and each node does one job. In the car, the LiDAR processor, camera processor, obstacle detection, planning and control are all separate nodes.

A · TOPIC



One publishes, many subscribe.

B · SERVICE



"Speed limit here?" → "50"

One request, one response.

C · ACTION



Goal → progress ... → result

Long tasks with updates.

PATTERN	USE IT FOR	CAR EXAMPLE	INSPECT IT
Topic	Continuous streams of data	LiDAR points on <code>/lidar</code> , images on <code>/camera</code>	<code>ros2 topic echo /lidar</code>
Service	Quick, one-off questions	Planner asks the map node for the speed limit	<code>ros2 service call ...</code>
Action	Long tasks that report progress	"Park the car", with updates until it's done	<code>ros2 action send_goal ...</code>

Messages

The typed data a topic carries, e.g.

`geometry_msgs/Twist` for velocity. See any type with `ros2 interface show`.

Parameters

Per-node settings you can change without recompiling, like a max speed. `ros2 param list`

Launch files

Start many nodes with their settings in one command: `ros2 launch pkg file.launch.py`

Packages *and* workspaces.

All ROS 2 code is organised and shared as **packages**. A **workspace** holds many packages, each in its own folder under `src/`.

A Python package

```
my_package/
  package.xml      # name + dependencies
  resource/my_package
  setup.cfg
  setup.py         # entry points for nodes
  my_package/      # your node .py files
```

A workspace

```
workspace_folder/
  src/
    cpp_package_1/
      CMakeLists.txt
      include/cpp_package_1/
      package.xml
      src/
    py_package_1/
      package.xml
      resource/py_package_1
      setup.cfg
      setup.py
      py_package_1/
  build/  install/  log/  # made by colcon
```

Create, build, run

```
# 1. make a workspace
mkdir -p ~/ros2_ws/src && cd ~/ros2_ws/src

# 2. create a package
ros2 pkg create my_package \
  --build-type ament_python \
  --dependencies rclpy

# 3. install dependencies
cd ~/ros2_ws
rosdep install --from-paths src \
  --ignore-src -y

# 4. build (from the workspace root)
colcon build --symlink-install

# 5. load it into this terminal
source install/setup.bash

# 6. run a node
ros2 run my_package my_node
```

The #1 beginner error: “Package not found”.
Open a new terminal? Run `source install/setup.bash` again. Add the ROS line to `~/.bashrc` so you never forget:
`source /opt/ros/<distro>/setup.bash`

Build only what changed

```
colcon build --packages-select my_package
```

The CLI *you'll actually use.*

For quick debugging and inspection. Keep this page open in a second window.

RUN THINGS

<code>ros2 run my_car lidar_node</code>	Start one node
<code>ros2 launch my_car car.launch.py</code>	Start several nodes at once
<code>ros2 pkg list</code>	Installed packages
<code>ros2 pkg executables my_car</code>	Nodes inside a package

NODES

<code>ros2 node list</code>	Running nodes
<code>ros2 node info /lidar_node</code>	Its topics, services, actions

TOPICS

<code>ros2 topic list -t</code>	All topics, with types
<code>ros2 topic echo /lidar</code>	Print messages as they arrive
<code>ros2 topic hz /camera</code>	How often messages arrive
<code>ros2 topic info /cmd_vel -v</code>	Who publishes / subscribes
<code>ros2 topic pub --once /cmd_vel \ geometry_msgs/msg/Twist \ "{lineat: {x: 0.2}}"</code>	Publish by hand

SERVICES, ACTIONS, PARAMS

<code>ros2 service list -t</code>	All services
<code>ros2 service call /srv type "{...}"</code>	Ask a question
<code>ros2 action list -t</code>	All actions
<code>ros2 action send_goal /a type "{...}" -f</code>	Send a goal, see feedback
<code>ros2 param list</code>	All parameters
<code>ros2 param set /node name value</code>	Change one live

TYPES AND RECORDING

<code>ros2 interface show <type></code>	Fields of a message
<code>ros2 bag record -a</code>	Record every topic
<code>ros2 bag play <folder></code>	Replay a recording

LOOK AT IT

<code>rviz2</code>	3D view of what the robot sees
<code>rqt_graph</code>	Live map of nodes and topics
<code>ros2 doctor</code>	Check your setup

Stuck? Tab-complete everything. `ros2 topic <Tab><Tab>` lists every option, and `-h` works on every command.

Tools, hardware *and* setup.

Five tools do most of the heavy lifting. Hardware is optional: everything in this guide can start in simulation.

RViz

See what the robot sees

LiDAR points, camera images, maps and paths in 3D.

rqt_graph

See how nodes connect

A live map of nodes and topics, so you can spot a missing link.

Gazebo

Test in a virtual world

A simulator for your robot before you use real hardware.

Nav2

Drive from A to B

A ready-made navigation system that moves a mobile robot while avoiding obstacles.

MoveIt

Move an arm safely

The Nav2 of robot arms: plans movements that don't hit anything, and does the inverse kinematics for you.

SLAM Toolbox

Build a map

Turns LiDAR scans into a map while the robot drives. Cartographer is the alternative.

Hardware (or use simulation)

Computer RPi or Jetson Orin Nano on the robot

Sensor A 2D LiDAR

Robot TurtleBot3, a ROS 2-supported AMR such as Waveshare's UGV Rover, or build your own

Arm SO-101: about \$200 to build, free in simulation

OS Ubuntu, ideally on x86. On a Mac, VMs also work well

Install checklist

- ☐ Install Ubuntu (native, dual boot or a VM)
- ☐ Install the ROS 2 distro that matches your Ubuntu version from docs.ros.org. Pick an LTS release
- ☐ Add `source /opt/ros/<distro>/setup.bash` to `~/.bashrc`
- ☐ Install `colcon` and `rosdep`
- ☐ Run `ros2 run turtlesim turtlesim_node`. If a turtle appears, you're ready

Same network, robots not showing up? Make sure every machine uses the same `ROS_DOMAIN_ID`.

The visual *roadmap*.

Seven projects, each one building on the last. Start on day one and fill in the gaps as you go.

PHASE 1 · MOVE		
00	Turtlesim The “Hello World!” of ROS. Move a turtle and watch messages flow over <code>cmd_vel</code> .	YOU LEARN Nodes, topics, testing your install
01	Teleoperate a real robot Drive an AMR with the keyboard, real or in Gazebo.	YOU LEARN Multiple nodes in multiple terminals, RViz
PHASE 2 · AUTONOMY		
02	Map a room with SLAM Build a map with LiDAR, then send Nav2 a goal.	YOU LEARN SLAM, A→B navigation, dynamic obstacle avoidance
03	Frontier exploration Your own algorithm and nodes: the robot maps the room on its own.	YOU LEARN Building on ROS 2, co-developing with a coding agent
PHASE 3 · INTELLIGENCE		
04	Semantic navigation A VLM labels what the robot sees, so you can send <code>/go_to_door</code> .	YOU LEARN VLMs, vector databases, semantic maps
05	LLM-controlled robot Command the robot in natural language through ROS-MCP.	YOU LEARN MCP, LLMs as planners for long tasks
PHASE 4 · MANIPULATION		
06	SO-101 arm with MoveIt A \$200 desk arm. Move to a 3D point, record and replay trajectories.	YOU LEARN Inverse kinematics, MoveIt, teleop, imitation-learning hardware

Move, teleop, *map*.

The fundamentals. Do these in simulation if you don't have a robot yet.

00 · TURTLESIM

```
# terminal 1: a blue window with a turtle
ros2 run turtlesim turtlesim_node
# terminal 2: drive it with the arrow keys
ros2 run turtlesim turtle_teleop_key
# terminal 3: watch the velocity commands flow
ros2 topic echo /turtle1/cmd_vel
```

Before you press a key, nothing is published on `/cmd_vel`. That's why the turtle sits still.

01 · TELEOPERATE A REAL ROBOT

```
# start keyboard teleop
ros2 run teleop_twist_keyboard \
  teleop_twist_keyboard

# in another terminal
ros2 topic echo /cmd_vel
```

The keys `i j k l` map to linear and angular velocities. Teleop publishes them to `/cmd_vel` and the robot's driver turns them into wheel speeds.

Your robot may use a different node or package name, so check its docs.

02 · MAP A ROOM WITH SLAM, THEN NAVIGATE

```
# 1. bring up robot + LiDAR
# 2. start mapping
ros2 launch slam_toolbox \
  online_async_launch.py
# 3. drive with teleop, watch RViz
# 4. save the map
ros2 run nav2_map_server \
  map_saver_cli -f my_room
# 5. navigate on it
ros2 launch nav2_bringup bringup_launch.py \
  map:=my_room.yaml
```

- ☐ Drive slowly while mapping. Fast turns blur the map
- ☐ In RViz, use **2D Pose Estimate** to tell Nav2 where the robot is
- ☐ Then click **Nav2 Goal** and the robot plans a path and drives there
- ☐ Put a box in its way and watch it re-plan

Explore, see, *talk*, *grasp*.

From here you stop only using ROS 2 tools and start building your own on top of them, with a coding agent writing the code.

03 · Frontier exploration

The robot maps the room itself

Frontiers are the edges between mapped and unexplored space. The loop:

1. Find frontier cells on the map
2. Group and sort them
3. Pick one as a goal and send it to Nav2
4. Repeat until no frontiers are left

It works like a robot vacuum mapping a new house. Describe the goal and constraints to Codex or Claude Code, then test and fix iteratively.

04 · Semantic navigation

“Go to the door”

An occupancy grid only knows free, occupied and unknown. To add meaning:

1. Capture images at regular intervals while exploring
2. A VLM labels objects and places
3. Store labels and coordinates in a vector database
4. Match a query by meaning and send the best coordinates to Nav2

Matching by meaning means “couch” still finds “sofa”.

05 · LLM-controlled robot

Talk to your robot

An MCP server bridges ROS 2 and an LLM client, so the model can read sensors and send commands. Connect the open-source ROS-MCP server to Claude, then try:

- ☐ “What topics are available?”
- ☐ “What’s the message type of /cmd_vel?”
- ☐ “Make a complete circle.”
- ☐ “Survey the room.” (a long-horizon task)

06 · SO-101 arm + MoveIt

A factory arm on your desk

1. Assemble the arm (allow 4–6 hours) or set up simulation
2. Calibrate leader and follower, get teleop working (LeLab gives you a GUI)
3. Build the ROS 2 package and connect MoveIt
4. Record trajectories, name them, and replay them yourself or through an LLM

By default the SO-101 runs LeRobot. ROS 2 adds deterministic control and MoveIt’s IK and planning.

Learn *on demand*.

You don't need months of courses, advanced math or years of programming first. Start on real projects from day one, get your hands dirty, and fill in the gaps as you go.

My AI workflow

- ☐ Write down what the robot must do and its constraints first
- ☐ Let a coding agent (Codex, Claude Code) write the code, ideally inside a harness
- ☐ Run it, then feed back the exact errors and `ros2 topic echo output`
- ☐ Iterate on small steps: one node, one behaviour at a time

When it doesn't work

- ☐ Did you `source install/setup.bash` ?
- ☐ Is the node running? `ros2 node list`
- ☐ Is data flowing? `ros2 topic hz /topic`
- ☐ Are names and types right? `rqt_graph`, `ros2 topic info -v`
- ☐ Using two machines? Same `ROS_DOMAIN_ID`

Every link

ROS 2 docs + install	docs.ros.org
Turtlesim tutorial	docs.ros.org · turtlesim
TurtleBot3 in Gazebo	docs.robotis.com
TurtleBot3	turtlebot.com
Waveshare UGV Rover	waveshare.com
Build your own robot	articulatedrobotics.xyz
Gazebo	gazebo.org
SLAM Toolbox	github.com/SteveMacenski
Nav2	docs.nav2.org
MoveIt	moveit.ai
ROS-MCP server	github.com/robotmcp
SO-101 + LeRobot	huggingface.co/docs/lerobot

FREE · LIVE · 5 DAYS · 10 PEOPLE

Want to build these together?

I'm running a free live 5-day ROS 2 bootcamp once at least 10 people are interested: projects in more detail, doubts answered, and a community to learn with. If you ticked the box on the form, you'll hear first.